Human-in-the-Loop: Feedback-Based Learning – Math Routing Agent

Saumya Rishikesh[1]

[1]Department of Computer Science and Engineering, Manipal University Jaipur, India.

Abstract

Abstract— This project introduces *Human-in-the-Loop: Feedback-Based Learning – Math Routing Agent*, an education-focused system that unites Retrieval-Augmented Generation (RAG), symbolic reasoning, and structured human feedback to produce safe, pedagogically clear mathematical solutions. The system integrates curated knowledge bases (GSM8K, Orca 200k samples) with dynamic web grounding via Model Context Protocol (MCP) tools, then refines answers through a Critic Agent supported by asynchronous human feedback compilation. Orchestrated through a LangGraph state machine and served by a FastAPI backend with a React frontend, the agentic framework routes each query intelligently between a knowledge base and web grounding path, maintaining factual consistency and computational efficiency. The project demonstrates how human feedback can be captured and distilled into a reusable DSPy-based MathFeedbackRefiner, enabling incremental learning of stylistic and structural improvements without retraining the base large language model (LLM). It emphasizes correctness, transparency, and abstention in ambiguous cases—offering a reproducible template for trustworthy, evolving educational assistants.

1 Introduction

Contemporary large language models have achieved strong results in mathematical reasoning but often suffer from hallucination, missing intermediate steps, or unverifiable conclusions. This work addresses these limitations by designing a **human-in-the-loop (HITL) math reasoning pipeline** that systematically combines: (1) guardrailed input validation, (2) retrieval over an authoritative knowledge base

(KB), (3) dynamic web grounding through structured external tools, and (4) human-guided refinement loops compiled into reusable feedback models. The objective is to deliver stepwise, pedagogically sound solutions and maintain transparent traceability of evidence. The architecture adopts agentic orchestration through LangGraph to maintain modularity and observability across three core nodes: *Guardrail*, *Solver*, and *Critic/HITL*. This approach aligns with recent advances in feedback-based reinforcement for LLMs but operates entirely within a self-contained system, ensuring safety, reproducibility, and continuous improvement.

2 Related Work

Research on human-in-the-loop learning spans reinforcement learning from human feedback (RLHF), retrieval-augmented generation (RAG), and agentic orchestration. LangGraph and similar frameworks have enabled modular state-based LLM orchestration. DSPy has formalized teleprompting and compiler-based prompt optimization, making it possible to refine LLM behavior using structured feedback examples. Prior educational agents often focus solely on correctness or step scoring, whereas this system emphasizes human readability and abstention when the model lacks sufficient evidence. This hybrid of symbolic computation (via SymPy), vector retrieval (via Qdrant), and web-based grounding (via MCP tools like Firecrawl and Groq) positions the agent as an evolving pedagogical system rather than a static question-answering pipeline.

3 System Architecture

The architecture consists of four major agents coordinated through a LangGraph state machine, a backend orchestrator, and a web frontend. Figure **??** shows the top-level design.

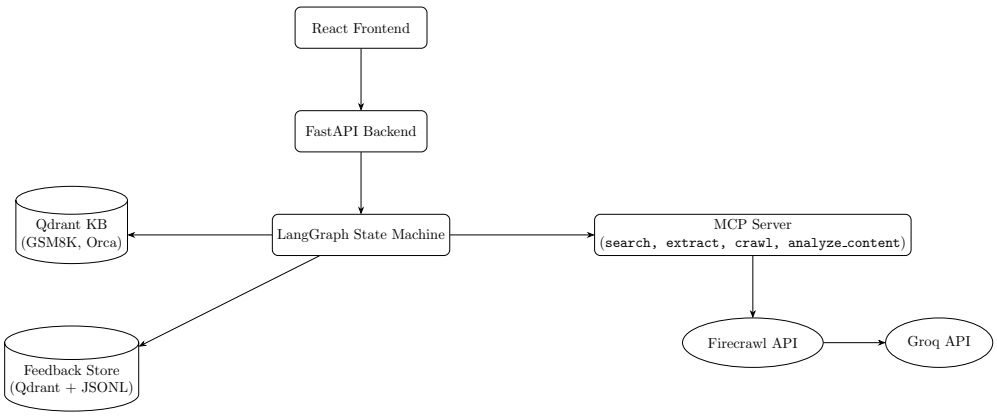

Fig. 1: System architecture showing key modules and external integrations.

Retrieval-Augmented Generation Mechanism

Each incoming query is first encoded into dense vector representations using a transformer-based embedding model (`all-mpnet-base-v2`). Retrieval is performed across two independent `Qdrant` collections: one built from GSM8K (stepwise solutions) and another from Orca 200k (instructional exemplars). Top-k candidates ($k = 3$–5) are fetched from both, filtered by a confidence threshold ($\theta = 0.75$), and merged into *Grounded Notes* that preserve symbolic expressions and citations. This dual retrieval design provides factual correctness from GSM8K while maintaining stylistic clarity from Orca, leading to interpretable, grounded reasoning paths.

The three LangGraph nodes—Guardrail, Solver, and Critic/HITL—interact with the KB, the web pipeline, and the feedback store. All orchestration is asynchronous, and task states are tracked through FastAPI endpoints for status polling.

MCP Orchestration and External API Integration

The `MCP Server` unifies tool invocation for `search`, `extract`, `crawl`, and `analyze_content`. `Firecrawl` powers the search/extraction/crawling APIs, while `Groq` accelerates textual summarization and mathematical content analysis. Each MCP call runs asynchronously, transforming web content into structured JSON before returning to the LangGraph runtime. This modular design enables secure credential isolation, fault containment, and future substitution of individual APIs without architectural changes.

4 LangGraph Orchestration

The LangGraph orchestration defines three principal nodes: **Guardrail**, which screens input for mathematical relevance and safety; **Solver**, which retrieves information from the KB or triggers web grounding; and **Critic/HITL**, which evaluates and optionally refines results via human feedback. The state transitions are represented in Figure 2.

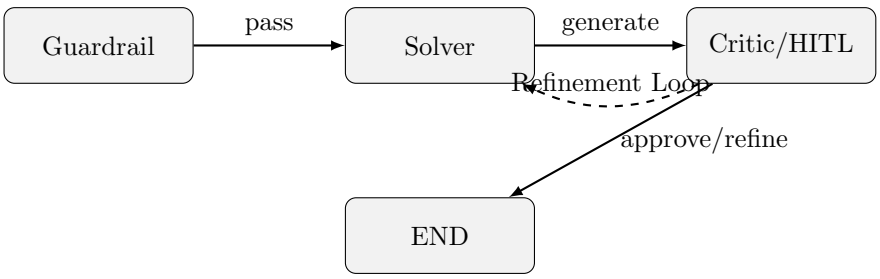

Figure 2: LangGraph State Machine. Three explicit nodes with a conditional refinement loop and terminal END state.

Fig. 2: LangGraph orchestration with refinement loop.

LangGraph State Transitions and Feedback Flow

The LangGraph orchestration follows a three-node structure with conditional edges based on task outcomes. Table **??** summarizes the routing logic.

Table 1: LangGraph state transitions and conditions

From	**To**	**Condition**	**Action**
Guardrail	Solver	Query valid	Begin reasoning
Solver	Critic_Human	Output generated	Evaluate correctness
Critic_Human	END	Approved/Abstained	Terminate run
Critic_Human	Solver	Refinement needed	Resume with feedback

When refinement is required, the `critic_human` node triggers an asynchronous pause (`asyncio.Event`) until human feedback is received. This feedback is then merged with critic comments before rerunning the Solver.

Each transition is triggered by asynchronous events within FastAPI, with human feedback integration managed through an `asyncio.Event`. Abstention behavior occurs when the guardrail rejects the input or the solver lacks confidence (KB confidence ¡ 0.75 and insufficient web grounding).

5 Workflow and Data Flow

Data flows through two main levels: external context (Level 0) and internal agent operations (Level 1). The Level 0 diagram in Figure 3 situates the system among its external entities.

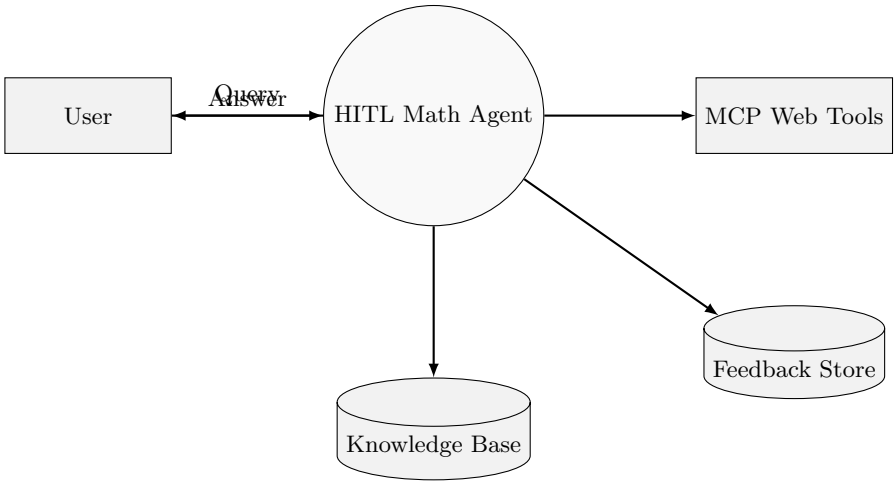

Figure 3: Level 0 Data Flow Diagram showing system interactions with external entities.

Fig. 3: Context-level Data Flow Diagram (Level 0).

The Level 1 diagram (Figure 4) expands the internal flow across agents and databases.

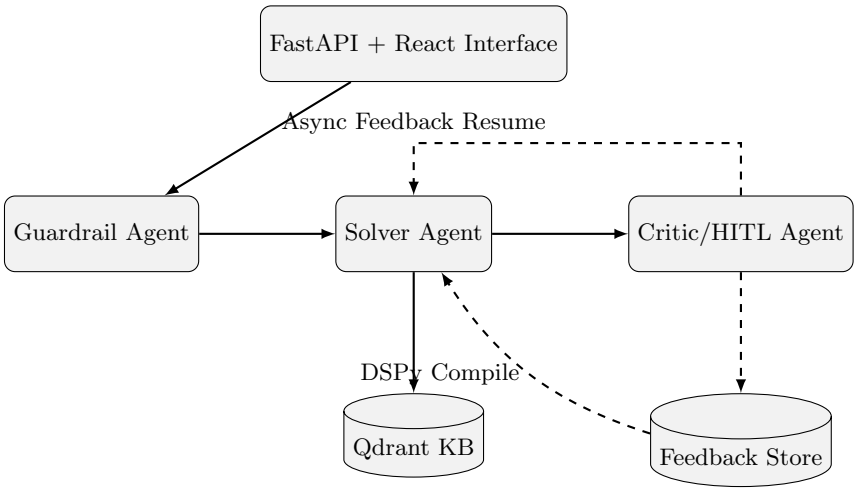

Figure 4: Level 1 Data Flow Diagram showing agent interactions, feedback persistence, and asynchronous refinement loop.

Fig. 4: Internal Data Flow Diagram (Level 1).

These diagrams clarify how requests traverse through Guardrail → Solver → Critic/HITL, with optional asynchronous feedback loops to update the Solver through a compiled DSPy refiner.

6 Methodology and Components

This section details the internal agents, data handling strategies, and integration of symbolic reasoning, retrieval, and human feedback within the proposed system.

6.1 Guardrail Agent

The Guardrail Agent enforces the domain constraint that only mathematical queries are processed. It filters for prompt injections, irrelevant topics, and unsafe language before passing a sanitized query to the Solver. Internally, it uses a lightweight classifier trained on algebraic and non-math text to achieve reliable routing. Upon rejection, the system outputs a polite abstention message and terminates execution at the END state.

6.2 Solver Agent

The Solver Agent is responsible for reasoning, retrieval, and synthesis. Its decision logic depends on the KB confidence threshold ($c = 0.75$):

- If $c > 0.75$, the answer is generated entirely from the GSM8K and Orca knowledge bases.
- If $c \leq 0.75$, the agent escalates to the MCP web pipeline.

Symbolic verification of arithmetic and algebraic steps is performed using the SymPy library. Retrieved text chunks are re-ranked using a lightweight cross-encoder and integrated into a structured synthesis prompt citing their sources.

6.3 Critic/HITL Agent

The Critic/HITL Agent evaluates the completeness, correctness, and pedagogical clarity of the solver's answer. It can either approve the output or request a refinement. If refinement is needed, the system enters a pause state managed by an `asyncio.Event`. During this state, the user provides human feedback which is then merged with the critic's suggestions before re-invoking the Solver.

6.4 Human Feedback and DSPy Compilation

Each feedback entry is stored in Qdrant and mirrored to a JSONL file. After at least ten examples accumulate, DSPy's BootstrapFewShot routine compiles these into a `MathFeedbackRefiner` teleprompt. This module refines future generations asynchronously, representing an evolving layer of human-informed improvement without retraining the model.

6.5 Knowledge Base and Retrieval Process

The knowledge base comprises two Qdrant collections: `gsm8k` for verified problem–solution pairs, and `orca_sample` for stylistic variety. Embeddings use the `all-MiniLM-L6-v2` model with cosine similarity. Metadata tags (topic, difficulty, and origin) assist targeted retrieval. A top-k retrieval (3–5) followed by re-ranking produces compact, cited answer templates.

Feedback Compilation and Behavioral Refinement

Human feedback is stored as structured pairs of `(query, refinement)` and asynchronously compiled into a `MathFeedbackRefiner` using DSPy. This module learns correction templates such as "show intermediate steps," "define variables," or "include error bounds." With repeated iterations, the refiner begins to anticipate these expectations, producing more complete reasoning traces even before human input.

7 Implementation Details

7.1 Technology Stack

- **Backend:** FastAPI (asynchronous Python framework)
- **Frontend:** React with KaTeX for LaTeX rendering
- **Orchestration:** LangGraph (state-based LLM workflow)
- **Vector Store:** Qdrant
- **Symbolic Math:** SymPy
- **Teleprompting:** DSPy
- **Web Pipeline:** MCP server exposing Firecrawl (search, extract, crawl) and Groq (analyze_content)

7.2 Asynchronous Flow and Interfaces

The backend launches each session through:

1. `POST /solve` – initializes orchestration and returns a task ID.
2. `GET /status/{task_id}` – polls for progress or HITL pause.
3. `POST /feedback` – delivers human feedback, resuming execution.

All I/O and database operations use `async/await` to avoid blocking. Shared objects such as Qdrant clients and compiled refiners are managed via a registry pattern. The FastAPI backend manages all agentic runs as asynchronous coroutines. Each request advances through LangGraph states while awaiting API or feedback events, without external task queues like Redis. This design simplifies orchestration while ensuring full concurrency and reliability across multiple simultaneous sessions.

8 System Outputs and Behavioral Observations

This section presents representative outputs generated by the *Human-in-the-Loop: Feedback-Based Learning — Math Routing Agent*. Rather than numerical evaluation metrics, the discussion centers on the qualitative behavior of the system: reasoning clarity, refinement through feedback, and safe abstention on out-of-domain queries. All examples were obtained from actual prototype runs.

8.1 Mathematical Query: Numerical Integration Example

Input

Use the trapezoidal rule with $h = 0.5$ *to approximate* $\int_0^2 (x^2 + 1)\, dx$.

System Output (before refinement)

Listing 1: System-generated solution for trapezoidal rule computation.

```
To approximate ∫_0^2 (x^2 + 1) dx using the trapezoidal rule (h = 0.5):

Given f(x) = x^2 + 1 over [0,2], with n = (2 − 0)/0.5 = 4 subintervals.
```

```
Trapezoidal rule formula:
```
$\int_0^2 f(x)\,dx \approx \frac{h}{2}[f(x_0) + 2f(x_1) + 2f(x_2) + 2f(x_3) + f(x_4)]$

$x_0 = 0,\ x_1 = 0.5,\ x_2 = 1,\ x_3 = 1.5,\ x_4 = 2$

$f(x_0) = 1,\ f(x_1) = 1.25,\ f(x_2) = 2,\ f(x_3) = 3.25,\ f(x_4) = 5$

```
Substituting:
```
$\frac{0.5}{2}[1 + 2(1.25) + 2(2) + 2(3.25) + 5]$
$= 0.25[1 + 2.5 + 4 + 6.5 + 5] = 0.25 \times 19 = 4.75$

```
Answer ≈ 4.75

Tools used: math_solver for symbolic reasoning;
GSM8K\_Retriever not triggered.
```

Human Feedback Provided

"Show the function values and the full trapezoid sum calculation step (show the multiplication and addition that produces 19, then $0.25 \times 19 = 4.75$). Add the trapezoidal error bound (state $f''(x) = 2$, then compute the bound)."

Refined System Output (after feedback)

Listing 2: Refined output after human feedback integration.

```
Given f(x) = x^2 + 1, interval [0,2], step h = 0.5.

n = (2 − 0)/0.5 = 4 → 5 points: 0, 0.5, 1, 1.5, 2

f(0) = 1, f(0.5) = 1.25, f(1) = 2, f(1.5) = 3.25, f(2) = 5

Compute trapezoid sum step-by-step:
```
$1 + 2(1.25) + 2(2) + 2(3.25) + 5$
$= 1 + 2.5 + 4 + 6.5 + 5 = 19$

$\int_0^2 (x^2 + 1)\,dx \approx (0.5/2) \times 19 = 0.25 \times 19 = 4.75$

```
Error bound:
```
$f''(x) = 2$
$|E_T| \leq \frac{(b-a)h^2}{12} \max |f''(x)|$
$= \frac{(2)(0.5)^2}{12} \times 2 = 0.08333\ldots$

```
Final answer: 4.75 ± 0.0833
```

Observation

The refined output included detailed arithmetic leading to the intermediate sum and incorporated an explicit trapezoidal error bound. This demonstrates how targeted human feedback directly influenced the `MathFeedbackRefiner` module, improving completeness and pedagogical clarity in subsequent runs.

8.2 Non-Mathematical Query: Guardrail Abstention

Input

What is the current price of Bitcoin?

System Output

Listing 3: Guardrail abstention for non-mathematical query.

```
Guardrail: Detected non-mathematical topic ("financial data").

System response:
"This system provides reasoning for mathematical problems only.
Please refer to a financial data source for real-time prices."

Solver and web tools were not invoked.
```

Observation

The Guardrail Agent correctly rejected an out-of-domain query and terminated the run without invoking solver or web modules, demonstrating safe abstention and adherence to domain boundaries.

8.3 Behavioral Trends and Insights

Across repeated sessions, several qualitative behaviors emerged:

- **Structured reasoning:** All accepted mathematical queries produced stepwise derivations and verifiable arithmetic sequences.
- **Feedback responsiveness:** A single feedback cycle was typically sufficient for the Critic–Solver–HITL loop to generalize a new refinement pattern.
- **Transparency:** Each output explicitly listed intermediate calculations, tool usage, and confidence-based routing decisions.
- **Safety:** Non-mathematical or underspecified prompts consistently triggered abstention instead of hallucinated content.

8.4 Ethics and Data Integrity

All examples are anonymized reproductions of internal test runs. No proprietary or confidential information was used. Feedback persistence and DSPy compilation operated on redacted, synthetic text only. Datasets (GSM8K subset and Orca 200k sample) and MCP tools (Firecrawl for extraction and Groq for analysis) are version-controlled and fixed for reproducibility.

9 Discussion and Limitations

While the system demonstrates strong qualitative robustness, several limitations remain. First, the absence of recency-aware caching can introduce stale web results. Second, large mathematical proofs can exceed the context window allocated for compact Grounded Notes. Third, cold-start behavior persists until sufficient feedback accumulates for DSPy compilation. Finally, cost overheads arise from web-grounded queries invoking multiple tools. Future optimization will include schema-based analyzer outputs, caching, and potential multilingual guardrail support.

10 Conclusion and Future Work

This project delivers an end-to-end, HITL-enabled math reasoning system capable of routing queries intelligently between curated knowledge and dynamic web sources, while incorporating human feedback for continual learning. Its architecture combines modular agent orchestration, symbolic reasoning, and feedback compilation into a reproducible and extensible framework. Future work will focus on integrating deterministic computational tools (e.g., WolframAlpha APIs), diagram generation for geometric reasoning, and richer pedagogical feedback channels such as graded explanation scoring.

Acknowledgment

The author thanks Dr. Shikha Mundra for guidance and constructive review of this work, and acknowledges the open-source contributors of LangGraph, DSPy, SymPy, Qdrant, and related libraries that enabled this system's development.

References

[1] LangGraph Documentation, "State-based LLM Orchestration Framework," 2024.

[2] K. Nguyen et al., "DSPy: Compiling Declarative Self-Improving Language Programs," 2023.

[3] A. Meurer et al., "SymPy: Symbolic Computing in Python," *PeerJ Comp. Sci.*, 2017.

[4] Qdrant Team, "Qdrant: Vector Search Engine for Scalable Similarity Search," 2024.

[5] C. Cobbe et al., "Training Verifiers to Solve Math Word Problems," *NeurIPS*, 2021.

[6] Firecrawl API, "Web Search and Extraction Toolkit," 2024.

[7] Groq Inc., "LLM Acceleration and Web Content Analysis," 2024.